BIENVENUE CHEZ NOWTECH

Manuel de l'équipe

Un guide pratique pour exécuter, modifier, valider et déployer le tableau de bord NowTech Admin.

COMMENCEZ ICI

1. Copiez .env.example vers .env
2. Ajoutez les valeurs locales requises
3. Exécutez bun install
4. Exécutez bun run dev:local
5. Ouvrez http://localhost:3000

En un coup d'œil

Framework

Nuxt 4, Vue 3.5, Nitro SSR

Gestionnaire de paquets

Bun, avec bun.lock comme seul fichier de verrouillage

Format de transmission

Archive de projet extraite

Ceci est un guide de travail, pas un remplacement d'AGENTS.md. Lisez ce fichier avant de modifier le code. Mis à jour le 6 octobre 2026 pour les contrôles de statut de stock et le dépôt du backend.

1. Comprendre le système

NowTech Admin est le tableau de bord opérationnel interne de NowTech. C'est une application Nuxt 4 qui effectue le rendu serveur des pages et expose un backend-for-frontend (BFF) Nitro. Le navigateur communique avec l'admin ; l'admin gère les appels authentifiés vers le backend NowTech et Supabase.



Carte du projet

Emplacement	Ce qui s'y trouve
app/	Pages Nuxt, composants, composables, plugins, schémas, stores et interface utilisateur du navigateur.
server/	Authentification réservée à Nitro, routes proxy du BFF, vérifications au démarrage et intégrations.
shared/	Types et fonctions utilitaires partagés entre app et server.
i18n/locales/	Domaines de traduction modulaires en anglais et en français.
test/unit and test/nuxt	Specs unitaires et de composants Nuxt à plat. Les motifs d'inclusion (globs) ne sont pas récursifs.

Frontière de sécurité

Ne déplacez jamais les jetons du backend dans le code du navigateur. L'accès à Supabase est créé à chaque requête dans server/utils/supabase.ts, et la session scellée n'accepte que son cookie. Le BFF utilise volontairement une liste blanche d'en-têtes transférés.

2. Installer et configurer

Prérequis

- Bun. La CI utilise Bun 1.4.0 ; utilisez une version locale compatible.
- L'archive du projet extraite et les valeurs de configuration fournies par l'équipe de transmission.
- Un backend NowTech accessible pour des données locales utiles et la connexion.
- Un fichier .env local. Gardez-le privé et hors des archives partagées.

Première exécution

```
cd <dossier-du-projet-extrait>
cp .env.example .env
bun install
bun run dev:local
```

bun install exécute Nuxt prepare via postinstall, ce qui crée .nuxt/. N'utilisez pas npm, pnpm ou yarn ici, même si l'ancien README à la racine les mentionne.

Configuration locale requise

Variable	Objectif et conseils locaux
NUXT_SESSION_PASSWORD	Au moins 32 caractères. Générez-la avec openssl rand -hex 32. Le serveur ne démarrera pas avec une valeur plus courte.
NUXT_SUPABASE_URL NUXT_SUPABASE_ANON_KEY	Point de terminaison Supabase et clé anonyme publique utilisés par le chemin d'authentification de production. Utilisez les valeurs fournies par l'équipe.
NUXT_API_BASE_URL	Origine du backend, sans /v1. Par exemple, un backend local peut être http://localhost:3001.
BACKEND_URL	Source pratique utilisée par .env.example ; NUXT_API_BASE_URL s'y réfère.

Règle sur les secrets : .env et .env.* sont ignorés, sauf .env.example et le faux hôte .env.test. Ne commitez jamais un fichier d'identifiants copié ou réel.

3. Exécuter avec un backend local

L'admin n'embarque pas le backend métier ; celui-ci vit dans son propre dépôt, décrit à la fin de cette section. Ce manuel suppose que vous disposez déjà d'une API en cours d'exécution et d'identifiants d'opérateur locaux valides. L'installation du backend, de PostgreSQL et des fixtures est couverte par le guide du backend lui-même.

Pointer le BFF vers l'API

```
BACKEND_URL=http://localhost:3001
NUXT_API_BASE_URL=${BACKEND_URL}
NUXT_LOCAL_AUTH_BYPASS=true
```

Démarrez ensuite le tableau de bord avec bun run dev:local et accédez à http://localhost:3000. Utilisez le compte opérateur local sur la page de connexion.

Ce que fait le contournement d'authentification local

1. Uniquement en environnement non-production, l'admin envoie l'e-mail de connexion et motDePasse à POST /v1/auth/connexion sur NUXT_API_BASE_URL.
2. Les rafraîchissements de jeton sont envoyés à POST /v1/auth/rafraichir.
3. L'application stocke un cookie de session scellé, et non un jeton backend exposé au navigateur.
4. Lorsque NODE_ENV vaut production, le contournement est indisponible et l'authentification Supabase est utilisée.

> **Lancez dev:local pour le backend local**
>
> bun run dev:local démarre le tableau de bord sur le port 3000 et efface les variables BACKEND_URL ou NUXT_API_BASE_URL exportées dans votre shell : ce sont donc les valeurs de .env, qui pointent vers votre backend local, qui s'appliquent. Avec bun run dev seul, une variable restée dans votre shell l'emporte sur .env, et l'application peut appeler un autre backend.

Valeurs optionnelles utiles

Variable	Quand la définir
NUXT_PUBLIC_STOREFRONT_URL	Origine publique de la vitrine utilisée par les liens produits. Elle est visible par le navigateur.
NUXT_PUBLIC_CATALOGUE_IMAGE_ORIGIN	Origine hébergeant les images de produits du catalogue.
NUXT_PUBLIC_LOGO_DEV_PUBLISHABLE_KEY	Aperçus de logo optionnels. Seule une clé publiable pk_ doit figurer ici, jamais un secret sk_ de Logo.dev.
NUXT_OPENAI_API_KEY	Clé IA de rédaction produit réservée au serveur. Optionnelle pour un travail local qui n'utilise pas la génération ; requise en production.
NUXT_PUBLIC_SUPPORT_EMAIL	Adresse de support optionnelle, visible par le navigateur, pour la page de connexion.

Le dépôt du backend

Le backend est le dépôt NowTech-backend-. Il est modifié en même temps que cette application et n'est plus considéré comme en lecture seule. Son guide, documents/travailler-et-deployer.md (rédigé en français), couvre l'installation locale, les tests et le déploiement. Ces règles comptent quand un changement de l'admin appelle un changement du backend :

Règle	Détail
Branche et pull request	Jamais de poussée directe sur main. Travaillez sur une branche et ouvrez une pull request qui dit ce qu'elle change et pourquoi.
Une fusion déploie	Fusionner dans main suffit pour livrer : GitHub prévient Coolify, qui reconstruit l'image et bascule le conteneur ; le changement est en ligne en deux minutes environ. Sur chaque pull request, GitHub exécute le typage, les migrations sur une base neuve, les fichiers sql/ appliqués deux fois, et la suite complète. Ne fusionnez pas une pull request rouge.
Migrations : jamais en arrière	Les migrations Prisma s'appliquent sur une base qui porte déjà des commandes et de l'argent. Écrivez-les pour qu'elles passent sur la production telle qu'elle est (add column if not exists) : une migration qui échoue empêche le conteneur de démarrer. Les fichiers sql/ doivent pouvoir être rejoués sans risque.
Aides locales	Dans le dépôt du backend, pnpm dev:jeton signe un jeton d'équipe pour appeler les routes d'administration sans Supabase. Avec SUPABASE_URL vide, le pnpm dev:local du backend fait de MOT_DE_PASSE_DEV le mot de passe de toute adresse, et un membre de boutique.membres_equipe prend ses rôles.

4. Boucle de développement quotidienne

Exécutez d'abord la plus petite vérification utile. Avant de partager ou de déployer un changement, exécutez les vérifications locales concernées.

Commande	À quoi ça sert
bun run dev:local	Démarre le serveur de développement sur http://localhost:3000, avec le backend défini dans .env.
bun run dev	nuxt dev simple, sans le nettoyage du shell : un BACKEND_URL ou NUXT_API_BASE_URL déjà exporté dans votre shell l'emporte sur .env.
bun run build **bun run preview**	Compile l'application SSR et sert la sortie de production en local.
bunx vitest run	Exécute les deux projets Vitest une fois. Utilisez bun run test:unit ou bun run test:nuxt pour un seul projet. La configuration fixe les réglages de Nuxt Image : les variables d'image ou de Supabase de votre shell ne peuvent pas changer le résultat.
bun run lint	Exécute les vérifications de correction d'oxlint.
bun run format **bun run format:check**	Applique ou vérifie le formatage avec oxfmt. Ne formatez pas à la main autour de lui.
bun run typecheck	Exécute la vérification de types Nuxt/vue-tsc.
bun run i18n:check	Vérifie la parité des clés anglais/français et les chaînes d'interface brutes.
bun run fallow	Audite le code modifié. Utilisez toujours bunx fallow ou les scripts Bun, jamais un binaire fallow global.

Tests et portes de qualité

Avant de partager un changement

Exécutez lint, format:check, i18n:check, fallow, typecheck, les tests et la compilation selon le changement.

Automatisation de votre équipe

Mettez en place votre propre CI et votre politique de revue pour reproduire les vérifications locales. Cette transmission laisse l'automatisation à la charge de la nouvelle équipe.

Le TDD est la norme par défaut : pour une fonction, un composant ou une règle métier, écrivez d'abord un test significatif qui échoue, puis faites-le passer. N'ajoutez pas de tests uniquement pour la couverture, uniquement des snapshots, ou qui testent le framework.

Base de référence connue des tests

bunx vitest run signale actuellement 12 tests en échec dans six fichiers : les quatre specs ai-product-copy, overview-date-field et products-detail-form. bun run typecheck signale une erreur, dans ai-product-copy-service. Tous précèdent le travail sur le statut de stock et sont consignés dans reports/2026-10-05-assessment-vitest-failures.md et reports/2026-10-05-assessment-typecheck-failure.md. Quelques specs Nuxt lourdes peuvent aussi dépasser leur délai quand toute la suite tourne sous charge ; relancez le fichier seul avant d'y voir une régression.

5. Comment nous modifions le code

Règles de travail

Domaine	Règle de travail
État serveur	Utilisez TanStack Query. Pinia est réservé à l'état propre au navigateur, comme les préférences et les filtres de brouillon, pas un miroir des données de l'API.
Composants	Créez un dossier par composant. Ne modifiez pas app/components/ui/ sauf demande explicite de l'utilisateur ; il s'agit d'une sortie générée par shadcn-vue.
i18n	Utilisez NuxtLinkLocale pour les liens internes et des routes localisées personnalisées. Les petits composants stables peuvent utiliser des messages par composant.
Styles	Utilisez les utilitaires Tailwind et les tokens CSS existants. Tailwind est configuré dans app/assets/css/main.css, pas dans un fichier tailwind.config.
Tests	Les specs unitaires s'exécutent sous Node ; les composants utilisent Nuxt + happy-dom. Gardez les fichiers directement dans test/unit ou test/nuxt, car les motifs d'inclusion sont à plat.

Flux de livraison

1 Planifier -> 2 Tester -> 3 Implémenter -> 4 Revoir

Un plan, une implémentation et une revue partagent un même slug en kebab-case. Les plans vont dans plans/, les comptes-rendus d'implémentation dans walkthroughs/, et les revues dans reviews/. Les investigations sans plan vont dans reports/.

Gardez chaque changement ciblé, expliquez sa validation et préservez le travail des autres contributeurs.

6. Statut de stock d'un produit

Deux choses décrivent le stock d'un produit : la quantité comptée, et le fait que la boutique le vende ou non. Depuis le 6 octobre, elles sont séparées. L'onglet Aperçu place le délai de livraison en premier, puis les deux contrôles en dessous, côte à côte ; l'onglet Stock porte le compte et ses mouvements.

Les deux contrôles

Contrôle	Ce qu'il fait
En stock / En rupture	Le statut de vente. En rupture masque le produit de la boutique et fait refuser les nouveaux paniers. Tant qu'un produit suit sa quantité, l'interrupteur est verrouillé, car c'est le compte qui décide.
Suivre la quantité	Activé : le produit est compté et obéit à son compte. Le désactiver conserve le statut du jour comme décision explicite, donc les clients ne voient aucun changement ; le réactiver rend la décision au compte. Un produit jamais compté ne peut pas être suivi tant que vous ne l'avez pas compté avec le crayon à côté de la quantité.
Badge de statut	Indique qui décide : Suit le compte, Depuis WooCommerce ou Défini dans le panel. Ce dernier est affiché en ton d'avertissement, car une décision du panel l'emporte sur chaque import WooCommerce.
Badge Non compté	Signale un produit qui n'a jamais été compté. Sa quantité affiche 0, atténué.

Cas limites

Situation	Ce qui se passe
Statut en contradiction avec le compte	Une boîte de dialogue d'avertissement cite le compte, par exemple quand on masque un produit qui a 7 unités en stock. Rien n'est écrit tant que l'équipe n'a pas confirmé.
Suivre WooCommerce	Un bouton de l'onglet Stock, proposé uniquement pour un produit non compté dont le statut a été fixé dans le panel. Sa boîte de dialogue nomme le statut abandonné ; Confirmer efface la décision, et la boutique utilise de nouveau le statut de WooCommerce, dès maintenant et après chaque import. Un produit compté rend la décision avec Suivre la quantité.
Nouveau produit	La quantité démarre vide. Laissez-la vide pour vendre sans compter, et choisissez un Statut (En stock par défaut) ; saisissez une quantité pour compter le produit et laisser la quantité décider.
Commandes déjà passées	Un statut En rupture fixé dans le panel masque le produit et refuse les nouveaux paniers, mais l'équipe peut encore confirmer une commande que le client a passée avant. Un produit en rupture parce que son compte est à zéro, ou dont le compte ne couvre pas la commande, bloque toujours la confirmation.
Marquer en réapprovisionnement	Masqué pour un produit jamais compté, et tant qu'une décision du panel est en vigueur.

Contrat avec le backend

La décision vit dans sa propre table, boutique.produit_disponibilites, et s'écrit par PATCH /v1/catalogue/administration/produits/:id/disponibilite. L'appel est réservé à la direction, ne crée jamais de ligne de stock ni de mouvement, et s'inscrit au journal sous stock.disponibilite_modifiee. La table est hors de ce que remplace l'ingestion WooCommerce : une décision survit à une réingestion.

7. Règles qui mordent

Ces contraintes sont délibérées. Traitez-les comme des limites de conception, pas comme des occasions de nettoyage.

Faites ceci	Parce que
Gardez zod à ^3.25.76	nuxt-zod-i18n dépend des exports runtime de zod 3. Zod 4 provoque un échec d'import en production sur Nitro.
Gardez TypeScript à ^5.9	vue-tsc 3 résout un chemin TypeScript que TypeScript 7 n'exporte plus.
Gardez createAuthClient() comme fabrique	Un singleton peut conserver une session en mémoire et faire fuiter le contexte d'une requête vers une autre.
Gardez sessionHeader: false	Cela empêche un en-tête x-nt-session de contourner la frontière de session limitée au cookie.
Utilisez la liste blanche d'en-têtes du BFF	proxyRequest transférerait le Cookie scellé par défaut. Les en-têtes de proxy explicites protègent cette frontière.
Ne préchargez que les données visibles sans défilement	Le limiteur partagé du backend est de 10 requêtes par seconde. Un tableau de bord SSR à six requêtes est une erreur de conception.
Conservez le détail des erreurs du backend	Les détails du backend, en français, restent affichés sous les titres de statut traduits, car le backend ne fournit pas encore de code d'erreur stable.
Fixer le statut par la décision, pas par le compte	N'écrivez jamais un mouvement ni une ligne de stock juste pour basculer un statut. Utilisez le point d'accès disponibilite, afin que le registre continue de n'enregistrer que des comptages.
Écrire les clés de traduction en appels t() littéraux	scripts/check-translation-keys.mjs, lancé par i18n:check, ne peut pas vérifier une clé construite à partir d'une chaîne de gabarit.

Sécurité de l'API

nuxt-security désactive son validateur XSS uniquement sous /api/** car il peut réécrire des valeurs JSON valides. La validation du backend reste la frontière d'entrée. N'élargissez pas cette exception.

Si une instruction semble inattendue, trouvez sa justification dans AGENTS.md ou les post-mortems liés avant de la modifier.

8. Compiler et déployer

Il s'agit d'une application SSR Nitro. Ne considérez pas nuxt generate comme le chemin de déploiement normal en production.

Compilation de production

```
bun install --frozen-lockfile
bun run build
node .output/server/index.mjs
```

Pour une vérification rapide en local après la compilation, exécutez bun run preview. Nitro respecte PORT ou NITRO_PORT, et HOST ou NITRO_HOST lors du service de la sortie.

Liste de vérification de la configuration de production

- Définissez les valeurs requises de session, de Supabase, d'origine API, d'image de catalogue et de vitrine depuis la section environnement.
- Définissez NUXT_OPENAI_API_KEY lors du déploiement. La vérification de configuration au démarrage l'exige en production, même si le développement local peut s'en passer.
- Gardez chaque secret réservé au serveur. Les valeurs publiques NUXT_PUBLIC_* sont volontairement exposées au navigateur.
- Définissez NUXT_BACKEND_SHARED_SECRET lorsque le backend est configuré pour reconnaître les requêtes de ce serveur en dehors de sa limite de débit partagée habituelle.
- Ne dépendez pas de NUXT_LOCAL_AUTH_BYPASS en production. Le code le bloque lorsque NODE_ENV vaut production.

Frontière de déploiement

Les éléments de transmission n'établissent ni projet d'hébergement, ni workflow de déploiement, ni configuration de domaine, ni chemin d'approbation, ni procédure de retour en arrière. Convenez de ces détails opérationnels avec la nouvelle équipe responsable avant toute mise en production. Ne les inventez pas.

Avant le déploiement

Confirmez que la compilation réussit, que l'environnement déployé dispose de tous les secrets requis, que l'origine du backend cible est correcte, et que le responsable du déploiement dispose d'un plan de retour en arrière. Alignez le modèle d'IA de rédaction produit et ses valeurs de coût d'entrée/sortie correspondantes avant de vous appuyer sur les chiffres de coût d'audit.

Le backend se livre séparément : une fusion dans sa branche main le déploie. Quand un changement de l'admin appelle un nouveau point d'accès du backend, comme le font les contrôles de statut de stock, le changement du backend doit être en ligne d'abord.

9. Où regarder ensuite

Utilisez ce manuel pour devenir opérationnel, puis consultez la source de vérité pour le détail.

Lisez ceci	Quand vous avez besoin de
AGENTS.md	L'ensemble des conventions du projet, la liste des commandes, l'architecture, les règles de test et les instructions spécifiques à chaque chemin.
.env.example	Les noms de variables actuels, les valeurs par défaut locales et des commentaires sur leur utilisation sûre.
package.json	Les commandes locales prises en charge pour la validation, le formatage, les tests et la compilation.
plans/, walkthroughs/, reviews/	Des précédents pour la planification, les notes d'implémentation et la profondeur des revues.
reports/2026-08-27-postmortem-zod-4-production-crash.md	Pourquoi zod doit rester sur la version pont 3.25.x prise en charge.
reports/2026-09-01-postmortem-node-26-localstorage-plugin-chain.md	L'échec de chaîne de plugins Node 26 / localStorage et son importance en production.
reports/2026-10-01-assessment-local-backend-merge-runtime-state.md	Le contexte connu du schéma du backend local, des fixtures, des références de commande et de l'exécution des bannières.
plans/ et walkthroughs/ 2026-10-06-decouple-stock-status.md	La conception, ses écarts, et les vérifications en conditions réelles derrière les contrôles de statut de stock.
NowTech-backend-/documents/travailler-et-deployer.md	Comment le backend se livre, ses règles de migration et de SQL, et le contrat du statut de stock. Rédigé en français.

Une dernière liste de vérification

1. Lisez AGENTS.md avant de toucher au code.
2. Utilisez Bun et gardez les identifiants uniquement dans .env.
3. Utilisez le chemin du backend local avec NUXT_API_BASE_URL et le contournement d'authentification non-production lorsque c'est approprié.
4. Écrivez d'abord des tests significatifs pour le comportement modifié.
5. Exécutez des vérifications ciblées, puis les portes complètes concernées.
6. Exécutez les portes locales concernées et consignez le résultat de la validation.
7. Demandez avant de faire des suppositions sur l'hébergement de production ou le backend jumeau.

Bienvenue dans l'équipe. Facilitez le prochain changement pour la personne qui vous suivra.